Technology Stack

Date	26/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	**Frontend / Client-Side (HTML, CSS, JavaScript)**	**HTML5, CSS3 (main.css), JavaScript (main.js), Jinja2 Templates**	HTML5 and CSS3 provide a responsive, dark-themed user interface with animated rain effects and a clean dark-navy design. JavaScript handles form validation and dynamic progress bar animations. Jinja2 templates (built into Flask) enable server-side rendering of prediction results directly into HTML pages without requiring a separate frontend framework.
2	**Backend / Server-Side (Python, Flask)**	**Python 3.8+ Flask 2.2+ Gunicorn Joblib**	Python is the leading language for machine learning and data science, providing seamless integration between the web application and ML model. Flask is a lightweight WSGI micro-framework that handles HTTP routing, form processing, and template rendering with minimal overhead. Gunicorn serves as the production WSGI server for deployment. Joblib efficiently loads the saved XGBoost model and StandardScaler at application startup.
3	**Database / Data Storage (File-based)**	**Excel File (flood dataset.xlsx) Joblib Save Files (floods.save, transform.save)**	No traditional relational database is required for this project. The flood prediction dataset is stored as an Excel file and used only during model training in Jupyter Notebook. The trained XGBoost model and fitted StandardScaler are serialised using Joblib and stored as .save files, which are loaded directly by the Flask application at runtime for real-time inference.
4	**Cloud / Hosting / Deployment (Render.com)**	**Render.com (Free Tier) Git-based Deployment**	Render.com provides a free cloud hosting platform that supports Python and Flask applications natively. It auto-deploys from the GitHub repository whenever new code is pushed, eliminating manual deployment steps. Render uses the Procfile to start the Gunicorn server automatically. It provides HTTPS, a public URL, and 99% uptime, making the flood prediction system accessible globally without any cost.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
5	**Version Control & CI/CD (GitHub, Git)**	**Git GitHub Procfile (Auto-deploy)**	Git provides distributed version control for tracking all code changes across the project lifecycle. GitHub hosts the remote repository and enables collaboration among all 5 team members. GitHub is directly linked to Render.com for continuous deployment — every push to the main branch automatically triggers a new deployment. The Procfile defines the startup command (gunicorn app:app) used by Render during deployment.
6	**ML Libraries & Data Science Tools (Third-Party)**	**NumPy, Pandas Scikit-learn XGBoost Matplotlib Seaborn Jupyter Notebook Anaconda**	NumPy and Pandas handle numerical computation and dataset manipulation. Scikit-learn provides preprocessing tools (StandardScaler, train_test_split, LabelEncoder) and baseline classifiers (Decision Tree, Random Forest, KNN). XGBoost delivers the best prediction accuracy of 96.55% using gradient boosting with class imbalance handling. Matplotlib and Seaborn generate all EDA visualizations including heatmaps, violin plots, and distribution charts. Jupyter Notebook and Anaconda provide the interactive development environment for EDA and model training.